

Frontend Engineer

Overview:

You will be responsible for implementing a **User Dashboard** in an Angular application that allows users to be added via a **lazy-loaded popup form**. The dashboard should display a **dynamic table** of users and a **Chart.js pie chart** representing the distribution of user roles. The chart and table should update in real time when a new user is added.

The components should be optimized using **lazy loading** for the form and chart to improve performance, and **RxJS** should be used to manage asynchronous state updates.

Detailed Task Breakdown:

1. Dashboard Setup:

- Implement a **UserDashboardComponent** that displays:
 - A **table** of users (with columns: Name, Email, Role).
 - A **Chart.js pie chart** showing the distribution of user roles (Admin, Editor, Viewer).
- Add an **Add User** button that will trigger the opening of a lazy-loaded user form modal.

2. Lazy Loading:

- Implement **lazy loading** for:
 - The **UserFormComponent** (the popup form for adding new users).
 - The **Chart.js** library and initialization (load dynamically when the component is initialized).
- Use Angular's module system to ensure the **UserFormComponent** is lazy-loaded only when required.

3. User Form (Popup):

- Create a **UserFormComponent** (lazy-loaded) with a form containing:
 - Name (text input)
 - Email (email input)
 - Role (dropdown: Admin, Editor, Viewer)
- Upon form submission, emit the new user data to the parent component (**UserDashboardComponent**) and close the modal.
- Ensure form validation for name, email, and role fields.

4. Data Management with RxJS:

- Implement **state management** for the list of users using **RxJS BehaviorSubject**.

- The **UserService** should manage adding new users and emitting updated user data.
 - The **UserDashboardComponent** should subscribe to the **users\$** observable and update the table and chart when new users are added.
5. **Dynamic Updates:**
- On adding a user:
 - Update the **table** with the new user data.
 - Update the **chart** showing the new distribution of roles (Admin, Editor, Viewer).
6. **Component Structure:**
- **UserDashboardComponent:** Displays the user table, chart, and controls the visibility of the modal.
 - **UserFormComponent:** Lazy-loaded modal form for adding users.
 - **UserService:** Manages user data and emits updates using RxJS **BehaviorSubject**.
-

Acceptance Criteria:

- **User Table:** The dashboard should display a table with columns for Name, Email, and Role. The table should be updated when a new user is added.
 - **Chart:** A dynamic pie chart (using Chart.js) should be displayed, showing the distribution of roles (Admin, Editor, Viewer). The chart should update when a new user is added.
 - **Modal Popup:** A modal popup (lazy-loaded) should appear when the **Add User** button is clicked. The form should allow the input of Name, Email, and Role, and should be validated before submission.
 - **Lazy Loading:** Both the **UserFormComponent** and **Chart.js** should be loaded lazily to improve the performance of the app.
 - **RxJS:** The state of the users should be managed via RxJS **BehaviorSubject**, and updates to the users should automatically reflect in the table and chart.
 - **No Errors:** There should be no errors or warnings in the console, and all required features should function correctly.
 - **Design-theme:** color code #383838, #1c4980 | Button, Input size: 48px
-

Technical Requirements:

1. **Technologies:**
- Angular (14+)
 - Chart.js (for pie chart)
 - RxJS (for state management and async operations)
 - Lazy loading for **UserFormComponent**

- **Code Quality:**
 - Ensure proper form validation for user input.
 - Ensure that the components are well-structured and follow Angular best practices.
 - Use **RxJS** to manage asynchronous state changes.
2. **Performance:**
- Ensure that lazy loading is correctly implemented to avoid unnecessary load at the start.
 - Optimize the update of the chart and table to ensure smooth performance.

Bonus Points:

- Add pagination or filtering functionality to the user table for better scalability.
- Enhance the user experience by adding loading indicators or transitions when data is being loaded or updated.